Forum Documentation

Team Members:

Lates Ovidiu-Alexandru
Boca Ioan-Doru
Blaga Alexandra

March 31, 2026

Contents

1 Technology Stack **1**

2 System Architecture **2**
2.1 API Gateway Layer 2
2.2 Microservices Architecture 2
2.2.1 Internal Microservice Layers 2

3 Persistence Layer (MongoDB) **3**
3.1 Core Data Abstractions 3
3.2 Infrastructure & Optimization 3

4 Communication Layer (gRPC) **4**
4.1 Messaging & Contract Layer 4
4.2 Handler Framework 4
4.3 Infrastructure 5

5 Execution Flow: Authentication Example **6**

1 Technology Stack

The project leverages a modern, distributed ecosystem designed for performance, scalability, and modularity.

- **Frontend**: React (for building a dynamic and responsive user interface).
- **Backend**: .NET 10 (C#), deployed within **Docker** containers to ensure consistent and isolated environments across development and production.
- **Database**: MongoDB (a NoSQL document database chosen for flexible and highly performant data persistence).
- **Key Frameworks & Libraries**:
  - *API Gateway*: `FastEndpoints` (for structuring RESTful routing with minimal overhead).
  - *Microservice Communication*: `Grpc.AspNetCore` and `Grpc.Tools` (for efficient, strongly-typed internal RPC calls).
  - *Data Access*: `MongoDB.EntityFrameworkCore` and the native driver.
  - *Security*: `System.IdentityModel.Tokens.Jwt` and `Microsoft.IdentityModel.Tokens` (for robust JWT-based authentication and authorization policies).

2 System Architecture

The system is designed using a modern distributed approach consisting of a central **API Gateway** and multiple specialized **Microservices**. This separation allows for independent scaling, technology flexibility, and clear boundaries between domains.

2.1 API Gateway Layer

The API Gateway acts as the entry point for all client applications, managing security, request routing, and protocol translation.

- **Routing & FastEndpoints**: Built using the FastEndpoints library, the gateway defines RESTful routes and translates incoming HTTP requests into gRPC calls for internal microservices.
- **Security & Policies**: Centralized authentication is handled via a custom Authentication Handler, which validates JWT tokens and enforces authorization policies such as Authenticated User or Moderator.
- **Response Aggregation**: A standardized base class manages how internal responses are returned to the client, mapping internal status codes to proper HTTP responses like 400 (Bad Request) or 404 (Not Found).

2.2 Microservices Architecture

Each microservice is an autonomous unit responsible for a specific domain (e.g., Users, Forum, Images) and follows a strictly layered internal structure.

2.2.1 Internal Microservice Layers

1. **Communication Level**: Handles gRPC interactions using a Message Dispatcher to route Protobuf requests to their specific logic handlers.
2. **Implementation & Business Logic**: Interfaces define the core capabilities, such as Authentication, Registration, and User Management, keeping the logic decoupled from the transport layer.
3. **Models & Persistence**: Contains database models and Data Transfer Objects (DTOs). Services utilize database abstractions for data persistence, often implementing optimized indexing for performance.

3 Persistence Layer (MongoDB)

The database layer is built around an abstraction of the MongoDB C# Driver to provide a clean, testable interface for business logic.

3.1 Core Data Abstractions

- **DatabaseModel**: An abstract record that enforces a standardized `ObjectId` representation for all persistent entities.
- **CollectionNameAttribute**: A custom attribute used to map C# classes to specific MongoDB collection names at runtime.
- **IDatabaseCollection**: A generic repository interface that provides standardized asynchronous operations such as `InsertAsync`, `UpsertAsync`, and predicate-based updates.

3.2 Infrastructure & Optimization

- **Database & Cache**: The `Database` class acts as a factory, caching collection wrappers in a `ConcurrentDictionary` to ensure thread-safe access and high performance.
- **AsyncLazy**: A utility class that uses `SemaphoreSlim` to ensure that heavy initialization tasks (like connecting to a collection and building indexes) are performed exactly once across multiple concurrent requests.
- **Dependency Injection**: An extension method registers the `IMongoClient`, `IMongoDatabase`, and the `IDatabase` singleton into the application container during startup.

4 Communication Layer (gRPC)

This project implements a Clean Architecture pattern specifically tailored for gRPC services in .NET. It utilizes a mediator-like pattern to decouple the gRPC service definitions from the actual business logic, ensuring that each request is processed through a standardized pipeline.

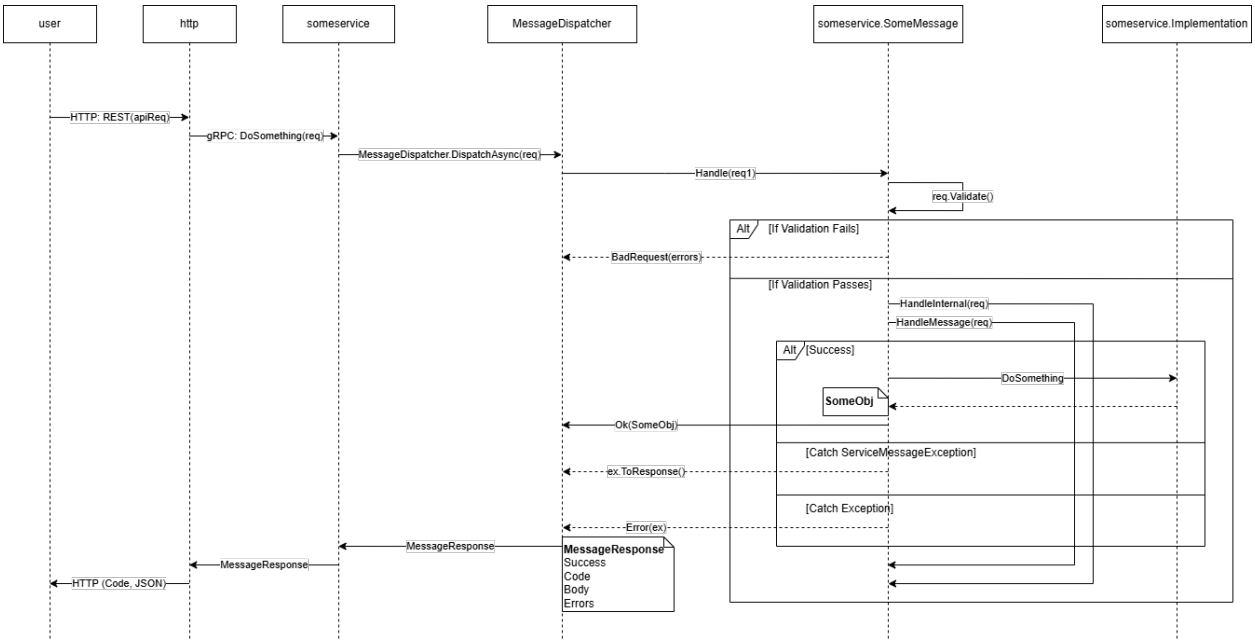


Figure 1: gRPC Request-Response Communication Flow

The sequence diagram above illustrates the request lifecycle. An incoming HTTP request from the client is translated by the Gateway into a gRPC message. This message is dispatched to the target microservice, where the `MessageDispatcher` dynamically resolves the appropriate handler. The base `Message` class intercepts the call to execute request validation, and then wraps the execution of the core business logic within a standardized `try-catch` block. Finally, the domain outcome (or exception) is packaged into a `MessageResponse` and propagated back to the API Gateway.

4.1 Messaging & Contract Layer

- **common.proto & MessageResponse.cs**: Define a unified internal communication contract across the entire ecosystem.
- **IRequestBase**: Ensures all internal requests are self-validating before they are processed by the business logic.

4.2 Handler Framework

- **MessageBase / Message**: Abstract classes that encapsulate cross-cutting concerns: validation, scoped logging, and standardized error handling.

- **ServiceMessageException**: A custom exception hierarchy that propagates errors from the business layer back to the gateway with specific semantic meaning.

4.3 Infrastructure

- **MessageDispatcher**: Decouples gRPC service definitions from their implementation, promoting highly readable and maintainable service classes.
- **MessageExtensions**: Provides automation for service registration using reflection to scan and register handlers into the dependency injection container.

5 Execution Flow: Authentication Example

The flow for authenticating a user demonstrates the integration of all layers in the system:

1. **Gateway**: Receives an HTTP POST request. The endpoint validates the payload and maps it to a gRPC request object.
2. **RPC Call**: The Gateway forwards the request to the Users Microservice via a generated gRPC client.
3. **Microservice Dispatch**: The Users Microservice receives the call and uses the Dispatcher to find the appropriate internal handler.
4. **Data Access**: The business logic handler uses an `AsyncLazy` collection reference to retrieve user data from MongoDB based on the provided email.
5. **Logic & Response**: The handler verifies the credentials, checks the user's ban status, generates a JWT, and returns a `MessageResponse`. The Gateway then maps this response into a standardized JSON object for the client.